Université de Monastir
Institut Supérieur d'Informatique et de Mathématiques de Monastir
Département d'Informatique

PROGRAMMATION PYTHON

Première Année Préparatoire Intégré
2025-2026

Séance 04

Rappel (Séance 3)

DataFrame

Manipuler la structure

- Afficher les dimensions , le nombre d'éléments, les informations, un résumé statistique, les noms des colonnes
- Renommer les libellés d'une ou de plusieurs colonnes
- Supprimer des lignes ou des colonnes

Afficher les données

- Afficher tout le contenu
- Afficher le contenu d'une cellule
- Afficher le contenu d'une ou de plusieurs colonnes
- Afficher le contenu d'une ou de plusieurs lignes

Manipuler les données

- Modifier le contenu d'une cellule ou d'une ligne
- Ajouter une colonne à un DataFrame
- Nettoyer le contenu (doublons, contenus vides par ligne ou par colonne)

8

Les fonctions statistiques sur les DataFrame

Moyenne

Nombre

Minimum

Maximum

Somme

8 Les fonctions statistiques

Pour commencer :

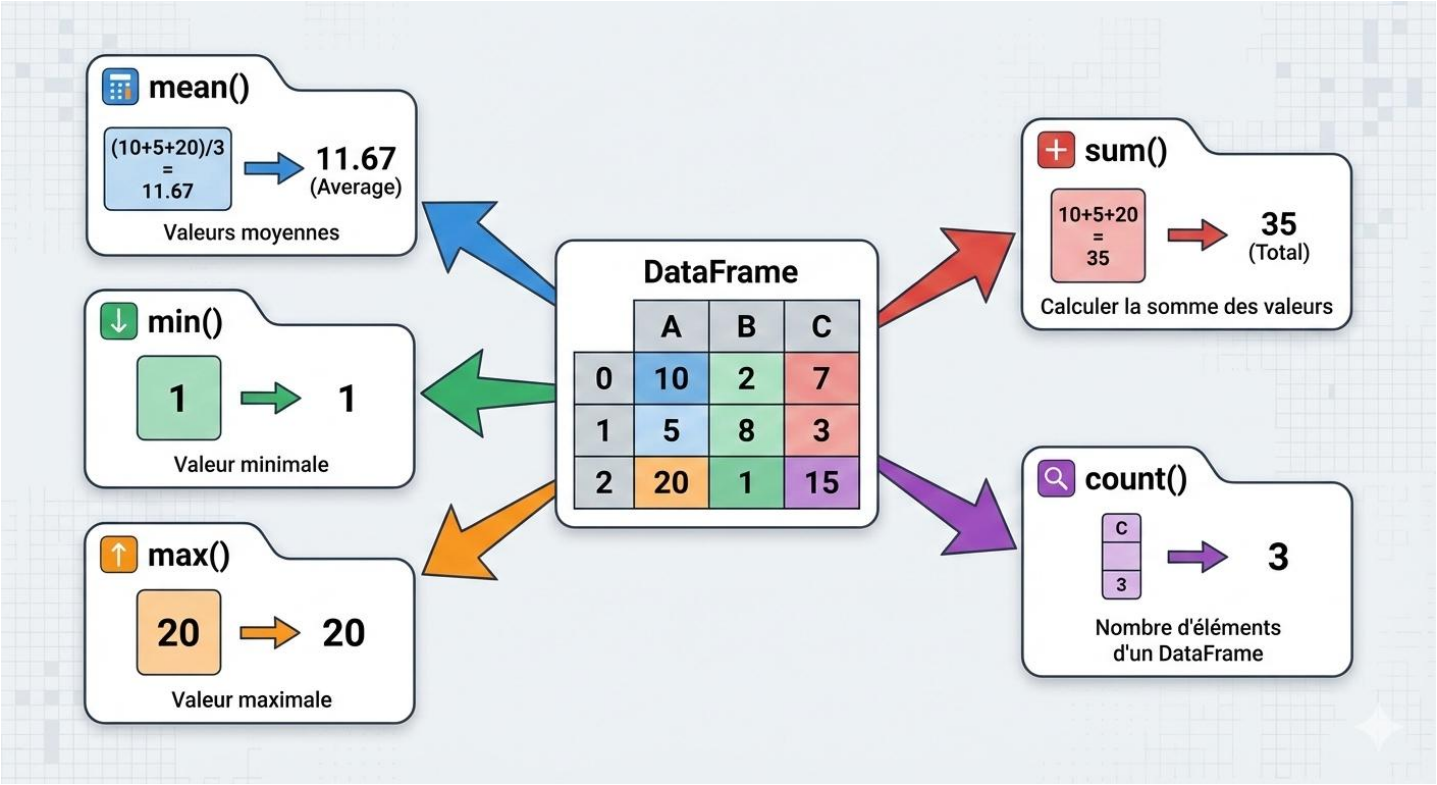
```
# Importation des bibliothèques
import pandas as ps

# Importation CSV
sante = ps.read_csv ("c:/Data/mesures.csv", sep=",")
print(sante)

# Nettoyage des lignes vides
sante = sante.dropna()
print(sante)
```

8 Les fonctions statistiques

Un DataFrame offre un certain nombre de fonctions statistiques telles que :



8 Les fonctions statistiques

Syntaxe générale sur un DataFrame :

```
print ( Id_DataFrame.Nom_Fonction() )
```

```
print(sante.mean())
print(sante.max())
print(sante.min())
print(sante.count())
print(sante.sum())
```

```
Duration      64.136905
Pulse        107.791667
Maxpulse     134.339286
Calories     375.675000
dtype: float64
```

8 Les fonctions statistiques

Syntaxe générale sur une colonne d'un DataFrame :

```
print ( Id_DataFrame ["Nom_Colonne"].Nom_Fonction() )
```

```
print(sante["Duration"].mean())
print(sante["Calories"].max())
print(sante["Pulse"].min())
print(sante["Maxpulse"].count())
print(sante["Calories"].sum())
```

```
64.13690476190476
1860.4
80
168
63113.4
```

8 Autres fonctions statistiques (à titre indicatif)

Catégorie	Fonction	Description
Produit	`Id_DataFrame.prod()`	Produit des valeurs
Indice Max	`Id_DataFrame.idxmax()`	Indice de la valeur la plus grande
Indice Min	`Id_DataFrame.idxmin()`	Indice de la valeur la plus petite
Médiane	`Id_DataFrame.median()`	Valeur centrale
Mode	`Id_DataFrame.mode()`	Valeur la plus fréquente
Écart-type	`Id_DataFrame.std()`	Dispersion autour de la moyenne
Variance	`Id_DataFrame.var()`	Carré de l'écart-type
Corrélation	`Id_DataFrame.corr()`	Relation entre variables
Covariance	`Id_DataFrame.cov()`	Variation conjointe

Activité 2

Question 1

Afficher la valeur moyenne des calories consommées.

```
print("La moyenne des calories est : ", sante["Calories"].mean())
```

Question 2

Afficher la valeur minimale et la valeur maximale des pulsations.

```
print("La pulsation minimale est : ", sante["Pulse"].min())
print("La pulsation maximale est : ", sante["Pulse"].max())
```

Question 3

Afficher la somme des calories consommées par les 10 premières personnes.

```
print("La somme est : ", sante.Calories[0:10].sum())
print("La somme est : ", sante.Calories.head(10).sum())
```

9 Fonctionnalités avancées

Trier un DataFrame

Afficher des données d’un DataFrame selon une ou plusieurs conditions

Ajouter une colonne remplie par des données suite à une condition simple

Exportation d’un DataFrame

Présenter des données à travers des graphiques

9 Tri d’un DataFrame

Tri par indices de lignes :

Resultat = Id_DataFrame.sort_index()

```
sante1 = sante.rename(index={170:180})
print(sante1)
sante1 = sante1.sort_index()
print(sante1)
```

```
169        60    115        145      310.2
180        75    120        150      320.4
171        75    125        150      330.4
172        75    125        150      330.4
```

```
169        60    115        145      310.2
171        75    125        150      330.4
172        75    125        150      330.4
180        75    120        150      320.4
```

9 Tri d’un DataFrame

Tri par colonnes :

Resultat = Id_DataFrame.sort_index (axis=1)

```
sante2 = sante.sort_index(axis=1)
print(sante)
print(sante2)
```

```
     Duration  Pulse  Maxpulse  Calories
0          60    110       130     409.1
1          60    117       145     479.0
2          60    103       135     340.0
```

```
     Calories  Duration  Maxpulse  Pulse
0       409.1        60       130    110
1       479.0        60       145    117
2       340.0        60       135    103
```

9 Tri d’un DataFrame

Tri des données d’un DataFrame :

```
Resultat = Id_DataFrame.sort_values ( by=["Nom_Colonne1" ,"Nom_Colonne2" , ...] ,
                                      ascending = [True/False , True/False , ... ] )
```

```
sante3=sante.sort_values(by=["Calories","Duration"], ascending=[False, True])
print(sante)
print(sante3)
```

```
     Duration  Pulse  Maxpulse  Calories
111       210    137       184    1860.4
81        270    100       131    1729.0
71        300    108       143    1500.2
..        ...    ...       ...       ...
102        20     95       112      77.7
95         15     80       100      50.5
91         20     83       107      50.3
[168 rows x 4 columns]
```

9 Affichage des données d'un DataFrame selon une conditions

Affichage des données d'un DataFrame selon une condition :

```
print( Id_DataFrame [ Id_DataFrame ["Nom_Colonne"] Op_Comp valeur ] )
```

LES OPERATEURS DE COMPARAISON					
==	<	<=	>	>=	!=

```
print(sante[sante["Calories"] >= 1000])
```

Affichage des données d'une ou de plusieurs colonnes selon une condition :

```
print(Id_DataFrame[["Nom_Colonne1", "Nom_Colonne2", ...]] [Id_DataFrame ["Nom_Colonne"] Op_Comp Valeur ] )
```

```
print(sante["Calories"][sante["Calories"] >= 1000])
print(sante[["Pulse", "Calories"]][sante["Calories"] >= 1000])
```

9 Affichage des données d'un DataFrame selon plusieurs conditions

Affichage des données d'un DataFrame selon plusieurs conditions :

```
print ( Id_DataFrame [ (Id_DataFrame["Nom_Colonne"] Op_Comp Valeur)
        Op_Logique (Id_DataFrame["Nom_Colonne"] Op_Com Valeur) ] )
```

LES OPERATEURS LOGIQUES		
Opérateur	Nomination	Rôle
|	OU logique	Vérifie qu'une des conditions est réalisée
&	ET logique	Vérifie que toutes les conditions sont réalisées

```
print(sante[(sante["Pulse"]>=85) & (sante["Pulse"]<95)])
print(sante[["Pulse","Calories"]][(sante["Pulse"]>=85) & (sante["Pulse"]<95)])
```

Activité 3

Question 1

Afficher le nombre de personnes ayant consommées plus que 400 calories.

```
print("Le nombre est : ", sante["Calories"][sante["Calories"]>400].count())
print("Le nombre est : ", sante.Calories[sante["Calories"]>400].count())
print("Le nombre est : ", sante.Calories[sante.Calories>400].count())
```

Question 2

Afficher la moyenne des calories consommées par des personnes ayant une pulsation comprise entre 110 et 115.

```
print("La moyenne est : ", sante["Calories"]
                                [(sante["Pulse"]>=110) & (sante["Pulse"]<=115)].mean())
```

9 Ajout d'une colonne remplie par des données suite à une condition simple

```
import numpy as alias
Id_DataFrame ["Nouvelle_Colonne"] = alias.where (
                    Id_DataFrame ["Nom_Colonne"] Op_Comp Valeur ,
                    Valeur_Si_Vrai ,
                    Valeur_Si_Faux )
```

```
import numpy as np
sante ["Etat"] = np.where (sante["Calories"]>=400, "Elevée", "Normale")
print(sante)
```

9 Exportation d'un DataFrame en csv

```
Id_DataFrame.to_csv ( "chemin\nom_fichier.csv"  , sep = "séparateur" ,
                                index = True/False , encoding = "utf-8"/"utf-8-sig" )
```

sep : spécifier le séparateur (virgule, point-virgule, tabulation)

index : indiquer si les indices seront enregistrer comme colonne ou non
(True : Enregistrer / False : Ne pas enregistrer)

encoding : encoder les caractères spéciaux ('utf-8' ou 'utf-8-sig')

```
sante.to_csv ("health.csv", sep ="\t" , index = False , encoding ="utf-8-sig")
```

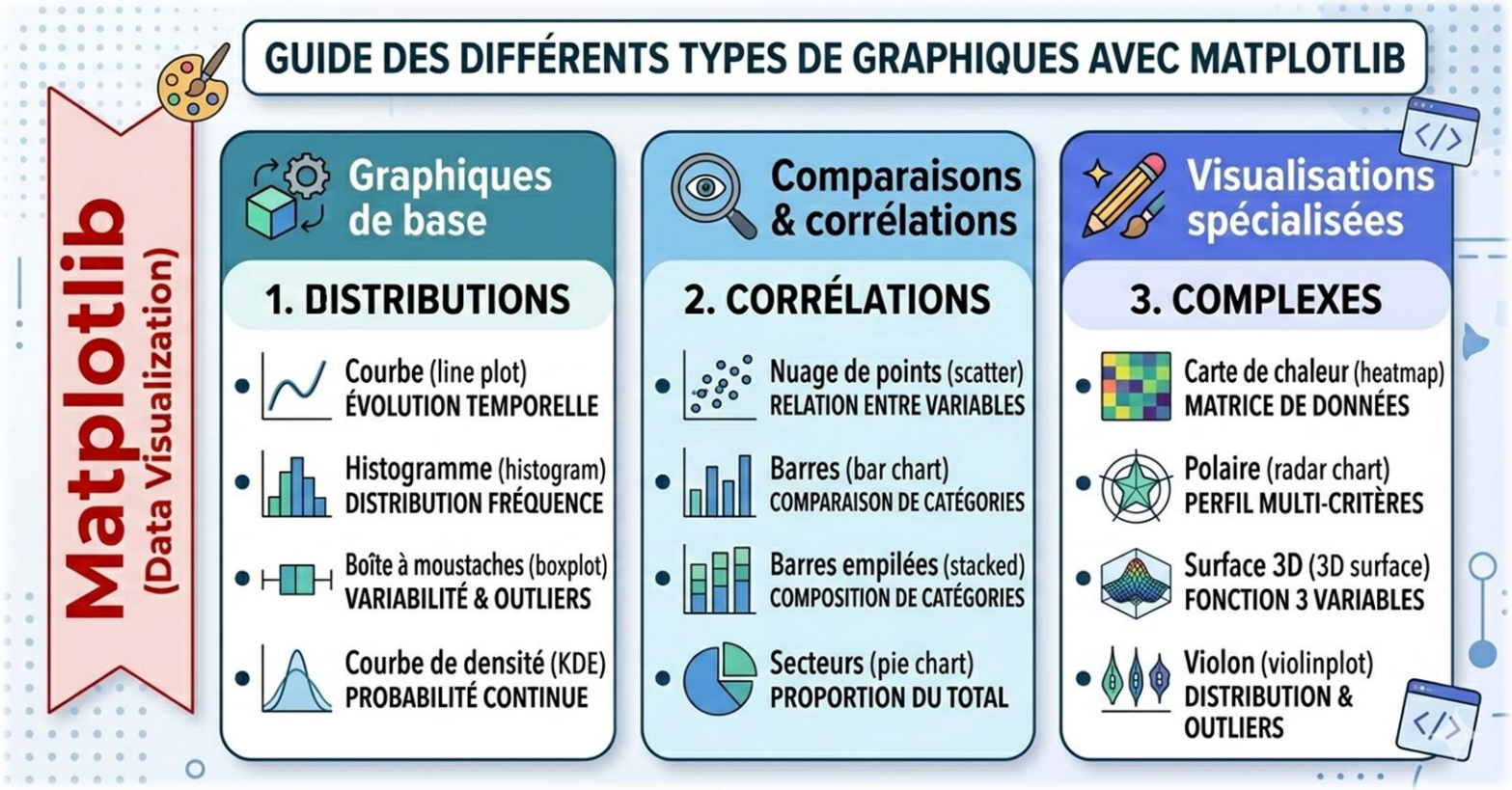
9 Présentation graphique des données

Pour générer des graphiques, il faut commencer, tout d'abord, par l'importation de la bibliothèque **matplotlib**.

```
import  matplotlib.pyplot as alias
```

```
import  matplotlib.pyplot as plt
```

9 Présentation graphique des données



9 Présentation graphique des données

Pour créer un graphique, on utilise la syntaxe suivante :

```
Id_DataFrame.plot.Type_Graphique ( x = "Nom_Colonne_Abscisse" ,
    y = "Nom_Colonne_Ordonnée" , title = "Titre_Graphique" ,
    color = "Couleur_Graphique" , xlabel = "Titre_Axe_Abcisse" ,
    ylabel = "Titre_Axe_Ordonnée" , legend = True/False ,
    figsize = (Longueur, Largeur) , rot = Degré_Rotation_Étiquettes )
```

9 Présentation graphique des données

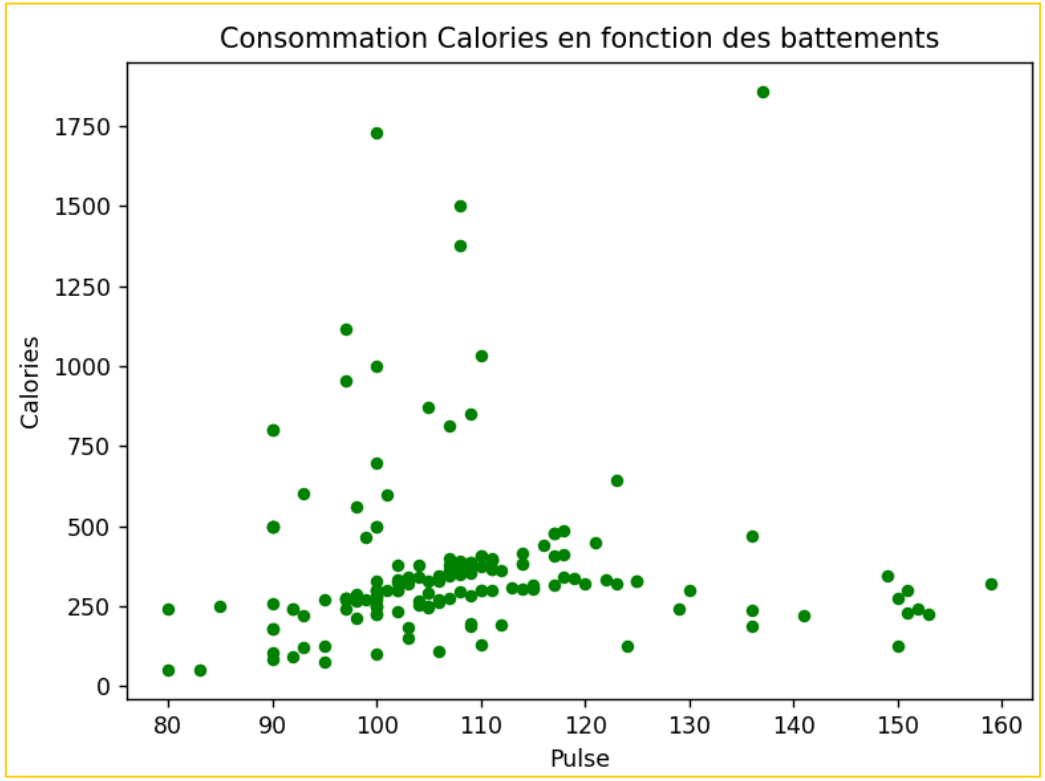
Type_Graphique: **line()** → courbe, **bar()** → barres, **scatter()** → dispersion
hist() → distribution, **pie()** → proportion

Pour ajuster automatiquement les paramètres du graphique pour que tous les éléments (titres, étiquettes, légendes, …) soient bien visibles sans chevauchement, on doit ajouter la fonction : **alias.tight_layout()**

Pour enregistrer le graphique sous format PNG, on doit taper la commande suivante : **alias.savefig ("Nom_Figure.png")**

Pour afficher le graphique, on doit taper la commande suivante : **alias.show()**

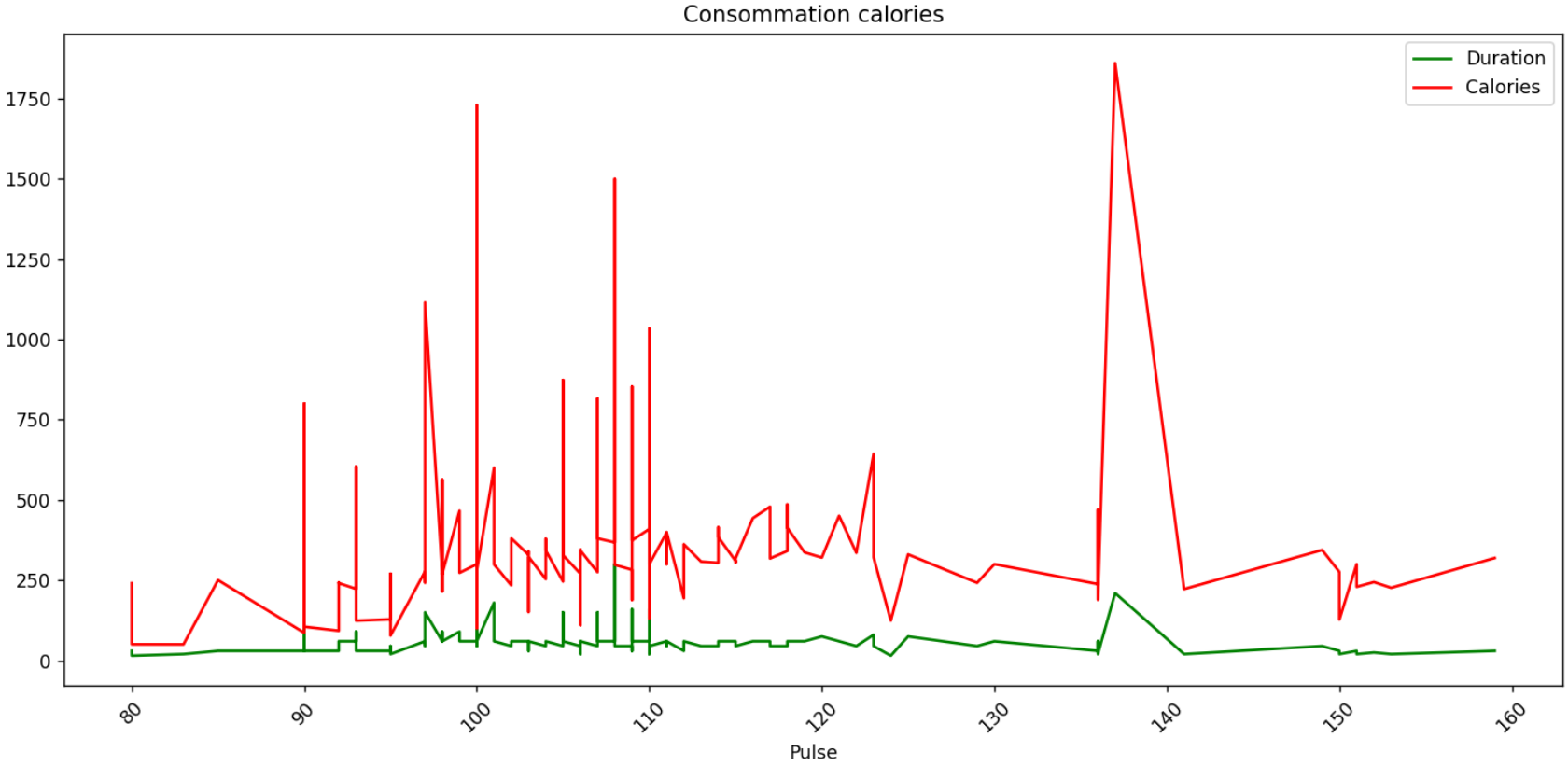
9 Présentation graphique des données



9 Présentation graphique des données

```
import  matplotlib.pyplot as plt
sante = sante.sort_values(by="Pulse", ascending=True)
sante.plot.scatter ( x="Pulse", y="Calories",
            title="Consommation calories en fonction des battements",
            color="green")
plt.tight_layout()
plt.show()
```

9 Présentation graphique des données



9 Présentation graphique des données

```
import  matplotlib.pyplot as plt
sante = sante.sort_values(by="Pulse", ascending=True)
sante.plot.line ( x="Pulse", y=["Duration", "Calories"],
                    title="Consommation calories", color=["green", "red"],
                    legend = True, figsize=(12,6), rot=45)
plt.tight_layout()
plt.savefig ("Fig1.png")
plt.show()
```